

REPORT_ID · FSP-AUDIT-2026-08 | METHODOLOGY · STATIC + ARCHITECTURAL
| AUDITOR · INDEPENDENT

تقييم محايد لمحاكي الطيران Flight Simulator Pro

تدقيق هندسي معمق لجاهزية المشروع للمنافسة على المستوى الدولي،
يغطي جودة الكود، فيزياء الطيران، العولمة، إمكانية الوصول، الأداء، الأمان،
والهوية التجارية — مع نظام درجات كمي وخارطة طريق إصلاحية مرحلية.

github.com/Abd123454/flight-simulator-pro

OVERALL READINESS SCORE

NOT GLOBAL-READY

52/100

يحتاج إصلاحات جوهرية قبل الادعاء بالجاهزية الدولية

الملخص التنفيذي والحكم النهائي

قرار سريع للقارئ غير التقني مع شرح موجز للنتيجة الكمية

يُقدّم هذا التقرير تقييماً محايداً لمشروع **Flight Simulator Pro** — محاكي طيران على المتصفح مبني على Next.js و 16 و Three.js و TypeScript. الهدف المعلن للمشروع هو الوصول إلى مستوى دولي عالمي يضعه في مصاف المنافسين الجادّين في فئة محاكيات المتصفح. بعد فحص دقيق لـ **32 ملفاً** مصدرياً (5,898 سطر برمجي) وللأصول الفنية والوثائق والبنية التحتية، نخلص إلى أن المشروع يمتلك **أساساً معمارياً قوياً** لكنه يعاني من **ثغرات جوهرية** تمنعه حالياً من الادعاء بالجاهزية العالمية.

النتيجة الكمية الإجمالية: **52 من 100** — وهي درجة تُصنّف المشروع ضمن المستوى "تحت المتوسط" وفق منهجية التقييم المعتمدة في هذا التقرير. تعكس هذه الدرجة مشروعاً له طموح واستثمار حقيقي في الفيزياء والرسومات، لكنه يفشل في معايير أساسية لجودة الإنتاج: **عدم وجود اختبارات**، إيقاف قواعد الـ ESLint، **عدم وجود ملف ترخيص**، غياب كامل للعولمة (i18n)، تسريبات ذاكرة في مكونات أساسيين، وخلل وظيفي حرج يجعل مواصفات الطائرات الأربع متطابقة فعلياً في الفيزياء.

خلل وظيفي جوهري • CRITICAL FINDING

ملف `aircraft-config.ts` يعرّف مواصفات واقعية لكل طائرة (الكتلة، مساحة الجناح، الدفع، معدلات الدوران)، لكن **محرك الفيزياء physics.ts لا يقرأ هذه القيم إطلاقاً** ويستخدم ثابتاً عالمياً واحداً (PHYS) لجميع الطائرات. النتيجة: مقاتلة F-16 تطير بنفس ديناميكا طائرة الركاب 737. هذا يجعل ادعاء README بأن "كل طائرة لها فيزياء مميزة" ادعاءً غير صحيح.

من جهة أخرى، يُظهر المشروع قرارات هندسية ناضجة: **خطوة زمنية ثابتة** (fixed timestep) بحماية ضد spiral-of-death، تخصيص مسبق للمتجهات (scratch vectors) لتجنب Garbage Collection، **إدارة ظلال اقتصادية** (ظل واحد فقط بدقة 1024)، وضع **توافق** للكرات الرسومية المدمجة (Intel HD)، وإطار **عمل منظم** مع فصل واضح بين الفيزياء والمحرك والمشاهد والكاميرا. هذا المستوى من النضج غير شائع في مشاريع الهواة، ويوفر قاعدة صلبة للبناء عليها.

هذا التقرير يفضّل النتائج على **11 محوراً**، يمنح درجة لكل محور، يصنّف الإصلاحات حسب الأولوية (حرج / عالٍ / متوسط / منخفض)، ويقدم خارطة طريق زمنية من أربع مراحل (90 يوماً) للوصول إلى مستوى المنشور الدولي.

47

إصلاح موصى به

52/100

الدرجة الإجمالية

5,898

سطر برمجي

32

ملف مصدري مدقق

منهجية التقييم ومعايير الجودة المعتمدة

اعتمد التقييم على فحص ثابت (static analysis) شامل لكامل قاعدة الكود دون تشغيل التطبيق، مع مرجعية معايير الصناعة المعتمدة دولياً. تم استخراج المقاييس تلقائياً عبر فحص ملف بملف، ومطابقة الادعاءات الواردة في README مع الكود الفعلي للتأكد من الاتساق. لم تُجرِ اختبارات تشغيلية (runtime) في هذا الإصدار من التقرير؛ لذا فإن المشاكل المتعلقة بالأداء الزمني (frame-rate) أو السلوك التفاعلي مبنية على تحليل الكود فقط.

معايير الجودة المعتمدة كمرجعية هي: **WCAG 2.1 AA** لإمكانية الوصول، **OWASP Top 10** للأمان، **Google Web Vitals** للأداء، **ISO/IEC 25010** لجودة البرمجيات، **W3C i18n** للعولمة، و**Google Lighthouse PWA** للهوية التجارية. كل محور يُمنح درجة من 0 إلى 100 مع تصنيف لفظي مرافق (ممتاز/جيد/مقبول/ضعيف/حرج).

المعيار	المرجعية الدولية
جودة الكود والبنية المعمارية	ISO/IEC 25010 — Maintainability
سلامة TypeScript وال lint	Community TS-Strict baseline
دقة فيزياء الطيران	NASA / ERAU stability-derivative reference
العولمة ودعم اللغات	W3C i18n + ICU MessageFormat
إمكانية الوصول	WCAG 2.1 AA + ARIA 1.2
الأداء وحجم الحزمة	Google Web Vitals (LCP/CLS/INP)
الأمان والقانوني	OWASP Top 10 + MIT license compliance
الهوية والعلامة التجارية	Google PWA Checklist + Open Graph
التوثيق والانضباط الهندسي	IEEE 26512 — User documentation

بطاقة الدرجات التفصيلية حسب المحور

11 محوراً مع وزنه النسبي في الدرجة الإجمالية

#	المحور	الوزن	الدرجة	التقدير	الحالة
01	جودة الكود والبنية المعمارية	15%	65	مقبول	يحتاج تحسين
02	سلامة TypeScript وال lint	8%	25	حرج	إيقاف قواعد أساسية
03	دقة فيزياء الطيران	12%	55	مقبول	خياطة غير مربوطة

#	المحور	الوزن	الدرجة	التقدير	الحالة
04	العولمة ودعم اللغات (i18n)	10%	0	حرج	غياب كامل
05	إمكانية الوصول (Accessibility)	10%	30	ضعيف	تحت WCAG الحد الأدنى
06	الأداء وتحسين الحزمة	10%	62	مقبول	تحسينات جيدة، فجوات
07	الاختبارات و CI/CD	10%	5	حرج	صفر اختبارات حقيقية
08	الأمان والقانوني	8%	40	ضعيف	لا رخصة، لا CSP
09	الهوية والعلامة التجارية (PWA/SEO)	7%	28	ضعيف	لا manifest، لا OG
10	التوثيق والاندماج الهندسي	6%	55	مقبول	README جيد، فجوات
11	إدارة الذاكرة والموارد	4%	45	ضعيف	تسريبات في فلتين
الدرجة الإجمالية الموزونة		100%	52	تحت المتوسط • ليس جاهزاً للعالمية	

تفسير الدرجة • INTERPRETATION

درجة ≤ 80 = جاهز للنشر الدولي المباشر. 65-79 = جاهز شرط إصلاحات بسيطة. 50-64 = يحتاج عمل هندسي جوهري. أقل من 50 = لا يُنصح بالنشر التجاري. المشروع في الحافة العليا من الفئة الثالثة، لكن ثلاثة محاور (i18n، الاختبارات، الـ Lint) تسحب الدرجة للأسفل بقوة لأنها تُعتبر "بديهيّات" الإنتاج.

65/100 جودة الكود والبنية المعمارية

5,898 سطر موزعة على 32 ملفاً — البنية نظيفة لكنها تعاني من ملفات ضخمة وتسريبات ذاكرة

البنية المعمارية للمشروع منظّمة بشكل احترافي: فصل واضح بين طبقة **المكتبة** (`src/lib/flight-sim/`) التي تحتوي المنطق النقي (الفيزياء، الإنجازات، المهام، الطقس)، وطبقة **المحرك** (`src/lib/flight-sim/engine/`) التي تتعامل مع Three.js والكاميرا والصوت، وطبقة **العرض** (`src/components/flight-sim/`) التي تتعامل مع React. هذا الفصل الثلاثي يتبع نمط Entity-Component-System المبسّط وهو قرار معماري سليم يُسهّل الصيانة والاختبار.

لا يوجد ملف يتجاوز 800 سطر، لكن ملف `FlightEngine.ts` (721 سطرًا) يقوم بمهام متعددة: حلقة العرض، إدارة حالة المهام، حساب الـ IL، نشر الـ HUD، الإنجازات، والإقلاع من جديد. يُنصح بتقسيمه إلى `MissionManager` و

Airplane.ts تسريب ذاكرة في HIGH

عند تحميل نموذج GLB واستبداله بالـ geometry الإجمالي، تُزال الكائنات القديمة من المشهد عبر `group.remove(child)` لكن لا يُستدعى `geometry.dispose()` أو `material.dispose()`. هذا يعني أن كل تبديل للنموذج يُسرب ذاكرة GPU. كذلك الفئة `Airplane` لا تحتوي على أي دالة `dispose()`، `FlightEngine.dispose()` لا يستدعيها. النتيجة: عند الخروج من لعبة والدخول مجدداً، تتراكم نماذج الطائرات في الذاكرة.

أيضاً Airport.ts تسريب في HIGH

نفس النمط: لا يوجد `dispose()`. كل الأشكال الهندسية (المرج، الأضواء، المباني، المروحيات الساكنة) تُسرب عند `teardown`. المفارقة أن `Environment.ts` هو الوحيد من الفئات الثلاث الذي يحتوي على `dispose()` صحيح يجتاز كل الأطفال ويتخلص من الموارد. هذا يدل على وعي هندسي لكنه غير مُطبق بانتظام.

كل إطار GL استعمال MEDIUM

في `FlightEngine.ts:542-553` يتم استدعاء `gl.getParameter(VENDOR)` و `getExtension('WEBGL_debug_renderer_info')` كل إطار (60 مرة في الثانية). هذه المعلومات ثابتة ولا تتغير بعد إنشاء الـ context، ويجب تخزينها مرة واحدة في المُنشئ.

قابلة للسباق module-level حالة MEDIUM

`let compatMode = false` في نطاق الـ module في `FlightEngine.ts`، ودالة `setCompatMode()` تُعدل هذه الحالة. هذا النمط يُسبب مشاكل إذا تعددت نسخ المحرك، ويجعل من المستحيل اختبار المحرك بمعزل، ويجعل عقد "اضبط قبل الإنشاء" هشاً. الصحيح هو تمرير الـ compat mode كـ constructor parameter.

من الناحية الإيجابية، يُظهر الكود وعياً ممتازاً بالأداء: المتجهات والـ quaternions تُخَصص مسبقاً (scratch vectors) ولا تُنشأ في الحلقة، حلقة الفيزياء تستخدم fixed-timestep بحماية ضد spiral-of-death (سقف 30 خطوة لكل إطار، dt clamp عند 0.5 ثانية)، و `try/catch` حول خطوة الفيزياء يمنع انهيار اللعبة كاملة عند خطأ واحد. هذا مستوى نضج غير شائع.

سلامة TypeScript وقواعد ESLint

أخطر قرار هندسي في المشروع: إيقاف 24 قاعدة لينت دفعة واحدة

هذا المحور هو **الأضعف** في المشروع بأكمله، والسبب ليس في الكود نفسه بل في قرار تكويني صريح. ملف `eslint.config.mjs` يُعطّل 24 قاعدة لينت جوهرية بشكل صريح، من بينها القواعد التي تُعتبر "بديهيات" أي مشروع إنتاجي:

— مُعطّل، يسمح بـ `any` بحرية، مما يلغي فائدة TypeScript. ◆ `@typescript-eslint/no-explicit-any`

- ◆ `@typescript-eslint/no-unused-vars` — مُعطل. اكتشف فعلياً: استيراد `useState` غير مستخدم في `Menus.tsx:2`.
- ◆ `react-hooks/exhaustive-deps` — مُعطل. يؤدي إلى bugs صامتة في الـ `effects`.
- ◆ `prefer-const` — مُعطل.
- ◆ `no-unreachable` , `no-debugger` , `no-console` — كلها مُعطلة.
- ◆ `react/no-unescaped-entities` — مُعطل. قد يُسبب أخطاء JSX صامتة.

لا قيمة له lint script الـ • CRITICAL

نتيجة إيقاف هذه القواعد، أصبح تشغيل `bun run lint` عملية شكلية بلا قيمة — لن يُكتشف أي خطأ حقيقي. هذا يلغي آخر خط دفاع ضد الانحدار (regression). أي مساهم جديد لن يحصل على أي تنبيه. هذا أكبر خطر على قابلية صيانة المشروع.

من جهة `tsconfig.json`: تم تفعيل `strict: true` (إيجابي)، لكن `noImplicitAny: false` يُضعف هذا التفعيل. الـ `target` مضبوط على `ES2017` وهو محافظ جداً يمكن رفعه إلى `ES2020`. كما يحتوي الـ `config` على استبعاد لمجلدات `skills` و `mini-services` غير موجودة أصلاً في المشروع — مما يدل على أن الملف نُسخ من قالب دون تعديل.

عدد حالات `as any` الموجودة فعلياً في الكود قليل نسبياً (4 حالات فقط)، ولا يوجد أي `@ts-ignore` أو `@ts-nocheck` أو `@ts-expect-error` — وهذا إيجابي. لكن وجود بعض `! non-null assertions` (مثل `connMat.map!` في `Airport.ts:73`) يُضعف أمان الأنواع.

06 • FLIGHT PHYSICS

55/100

دقة فيزياء الطيران

نموذج `blade-element` جيد البنية لكنه غير مربوط بإعدادات الطائرات

ملف `physics.ts` (447 سطراً) يُظهر فوهماً حقيقياً للديناميكا الهوائية. النموذج يعتمد على **معاملات رفع/سحب** (CL/CD) مقابل زاوية الهجوم، **مشتقات الاستقرار** (`weathervaning`, `dihedral`, `pitch/roll/yaw damping`)، **تأثير الأرض** (`ground effect`)، **الانزلاق الجانبي** (β)، **ال stall**، و**المنسق للمنعطف** (`coordinated turn`). المرجع المذكور (ERAU stability textsg Jakob Maier's physics-based sim) هو مرجعية صحيحة. تخصيص 17 `scratch vectors` في نطاق الـ `module` = **صفر تخصيص في كل إطار**، وهو مثالي.

فجوة بين التكوين والتنفيذ • CRITICAL

ملف `aircraft-config.ts` يعرّف 4 طائرات ببيانات واقعية (كتلة، مساحة جناح، دفع، معدلات دوران، `stall` (AoA) لكن **محرك `physics.ts` لا يستخدم أيّاً منها**. بدلاً من ذلك يستخدم ثابتاً عالمياً واحداً (PHYS (قيم 737 لكل الطائرات. التحقق: `stepFlight(state, orientation, apControls, STEP, weather)` لا يستقبل `aircraftConfig`. النتيجة: F-16 تتصرف ك 737، و Extra 300 تتصرف ك 737. هذا يجعل الادعاء التسويقي "كل طائرة بفيزياء مميزة" غير صحيح.

حتى لو رُبطت الإعدادات، فإن بعض القيم في `aircraft-config.ts` غير دقيقة واقعيّاً:

الطائرة	الحقل	القيمة في الكود	القيمة الواقعية	التقييم
Skyliner 737	maxThrust	600 kN	242 kN	2.5 × فضّمْ
Falcon F-16	maxThrust	380 kN	127 kN (محرك واحد)	3 × خطأ فادح
Acro Extra 300	maxThrust	40 kN	~3 kN (مروحة)	13 × فضّمْ
Heavy C-130	maxThrust	500 kN	~70 kN (4× T56)	7 × فضّمْ
Skyliner 737	wingArea	124.6 m ²	124.6 m ²	مطابق
Falcon F-16	wingArea	27.87 m ²	27.87 m ²	مطابق
Heavy C-130	wingArea	162 m ²	162.1 m ²	مطابق

مساحات الأجنحة **دقيقة تماماً** (مأخوذة من المراجع الرسمية)، لكن قيم الدفع فضّمة بشكل كبير تحت ذريعة "arcade feel". الأكثر إشكالاً: **F-16 له محرك واحد** في الواقع، لكن التعليق في الكود يقول "2 × dry 76kN afterburner boost" — وهذا خطأ وقائعي.

MEDIUM · حالات حافة غير مُدخلة

لا يوجد نمذجة لـ **spin** (الدوران الحلزوني بعد الـ stall)، **لا ضغطاً صوتياً** (compressibility, Mach drag rise) — حتى لو طار F-16 بـ 550 m/s كما يحدد الـ config، لا يوجد سلوك فوق صوتي. لا يوجد **asymmetric thrust**، **لا احتراق وقود**، **لا إزاحة لمركز الثقل**. تأثير الأرض يُقلّل السحب لكن لا يزيد الرفع (كما في الواقع).

0/100 **العولمة ودعم اللغات (i18n)**

غياب كامل — وهذا يُلغي "العالمية" بشكل قاطع

هذا هو المحور الذي يُفشل شرط "العالمية" بشكل مباشر وصريح. المشروع **لا يحتوي على أي دعم للغات**: لا مكتبة i18n، لا ملفات ترجمة، لا توجيه لغوي، لا دعم RTL، لا تبديل وحدات. كل النصوص في الواجهة مكتوبة بشكل حرفي (hardcoded) بالإنجليزية داخل المكونات. لغة الـ HTML مبسوطة على `<html lang="en">` بشكل ثابت.

CRITICAL · المتطلبات لتحقيق العالمية

لتحقيق الجاهزية الدولية، يجب:

- إضافة `next-intl` أو `react-i18next`.
- استخراج كل النصوص (~120 نصاً) إلى ملفات ترجمة (`messages/ar.json`, `messages/en.json`، إلخ).
- إضافة توجيه لغوي (`/en/`, `/ar/`, `/fr/`, `/es/`).
- تحويل Tailwind إلى خصائص منطقية (`pe`, `ps` - بدل `pr`, `pl` -) لدعم RTL.
- إضافة مُبدّل وحدات (متري / إمبراطوري / بحري — عرف الطيران يستخدم قدم + عقدة).
- مُبدّل لغة في الواجهة.

المشكلة الثانوية: **وحدات القياس غير قياسية**. السرعة تُعرض بـ km/h والارتفاع بالأمتار، بينما **عرف الطيران الدولي** يستخدم قدم للارتفاع وعقدة للسرعة (knots). طيار حقيقي سيجد الواجهة مُربكة. يجب توفير تبديل بين النظم الثلاثة على الأقل، مع الافتراضي حسب لغة المستخدم.

أيضاً، تنسيق التواريخ والأرقام غير متسق: `score.toLocaleString()` يستخدم لغة المتصفح، بينما الواجهة إنجليزية دائماً — هذا تناقض. `new Date(a.unlockedAt).toLocaleDateString()` نفس المشكلة.

إمكانية الوصول 30/100

تحت الحد الأدنى لـ WCAG 2.1 AA — حاجز أمام المشتريات الحكومية والتعليمية

ما هو موجود: `aria-label` على الأدوات الأربعة (CompassTape, AltimeterTape, Minimap, AttitudeIndicator) — إيجابي. أزرار shadcn/ui تحتوي على focus ring. ال Slider من Radix له ARIA مدمج. لكن هذا لا يكفي.

غائب كلياً عن قارئ الشاشة HUD • HIGH

ال HUD مضبوط على `pointer-events: none` وكل عناصره `aria-label` جامد. قارئ الشاشة يقرأ "Compass tape" و "Altimeter tape" لكن **لا يصل إلى القيم** (السرعة، الارتفاع، الاتجاه). لا توجد مناطق `aria-live` لإعلان التحذيرات (ال stall، اقتراب الهدف). أحداث المهمة مرئية فقط.

aria-hidden إيموجي زخرفي بدون • HIGH

عشرات الإيموجي في الواجهة (🔒 ✅ 🏆 ⚡ ⚠️ 🎮 🦾 🌧️ ☁️ ⚙️) بدون `aria-hidden="true"`. قارئ الشاشة سيُعلن كل واحد منها، مما يُسبب فوضى سمعية. مثال: "sun 🌤️ cloudy 🌥️ rain 🌧️". يقرأ كـ "sun emoji 🌤️ emoji cloud emoji rain emoji".

اعتماد كلي على اللون • MEDIUM

أضواء الجناح (أحمر/أخضر)، أضواء الهبوط (أحمر/أخضر)، أضواء PAPI (أحمر/أبيض) — كلها **تميز باللون فقط**. 8% من الرجال لديهم عمى ألوان. لا يوجد بديل أيقوني أو نصي. يجب إضافة رموز شكلية (مثلث/دائرة) إلى جانب اللون.

prefers-reduced-motion لا دعم • MEDIUM

`animate-pulse` مُستخدم في تحذير ال stall وال combo وال gust و autopilot. لا يوجد `media@` لـ `reduce(reduce: prefers-reduced-motion)`. المستخدمون الذين يعانون من دوار الحركة سيُزعجهم ذلك.

♦ **لا روابط skip-to-content** للتنقل بلوحة المفاتيح.

♦ **لا عناصر <kbd>** دلالية لوصف مفاتيح التحكم — مجرد نص inline.

♦ **لا وضع تباين عال** (high-contrast mode).

♦ **لا captions** للإشارات الصوتية (هبوط، تحذير).

♦ **لا fallback نصي** لل canvas في حال فشل WebGL.

62/100

الأداء وتحسين الحزمة

تحسينات جيدة للـ GPU لكن غياب أساسي في code-splitting

المشروع يُظهر اهتماماً واضحاً بالأداء: نسبة البكسل مضبوطة على 1.0، ظل واحد فقط بدقة 1024، InstancedMesh، LOD، MSAA ولا post-processing، textures إجرائية ≥ 1024 بكسل، لا culling للكائنات البعيدة، تحديث HUD مقتصد على ~30 Hz. هذه قرارات صحيحة لاستهداف الكرات المدمجة.

HIGH • Three.js غير مقسّم

KB minified, ~150 600) **كاملة** في 6 ملفات `import * as THREE from 'three'` لكن `import *` يلغي هذه الفائدة. لا يوجد chunk واحد. Three.js يدعم tree-shaking لكن `import { Scene, WebGLRenderer } from 'three'` استخدام `experimental.optimizePackageImports` في `next.config.ts`. الحل: استخدام `named imports`.

MEDIUM • GLB لنماذج preload لا

النماذج (KB 400) تُحقل عند بداية الطيران فقط، مما يُظهر geometry إجرائي لـ 100-500ms قبل ظهور النموذج الحقيقي. يجب preload أثناء شاشة اختيار الطائرة.

MEDIUM • كل إطار renderer.info

`renderer.info.render.calls` و `renderer.info.render.triangles` و `info.memory` تُقرأ كل إطار لإحصائيات الـ FPS. هذا يُجبر الـ renderer على تحديث إحصاءاته — يكلّف أداءً.

❖ لا `powerPreference: 'low-power'` — يستهلك بطارية اللابتوبات بقوة.

❖ غيوم (mesh 80~) بدون InstancedMesh — فرصة تحسين كبيرة.

❖ أدوات HUD تعيد الرسم كل تحديث (~30 Hz) حتى لو لم تتغير القيم. `React.memo` بـ comparator يُحسن الموقف.

❖ لا `ResizeObserver` على حاوية المحرك — تغيير اتجاه الجوال لن يعيد الحجم.

10 • TESTING & CI

5/100

الاختبارات و CI/CD

صفر اختبارات حقيقية — ثاني أكبر ثغرة في المشروع

مجلد `tests/` يحتوي على 3 سكريبتات `shell`: `python-runtime-build.sh`، `python-runtime-container.sh`، `database-runtime-build.sh`. هذه السكريبتات **لا علاقة لها بمحاكي**

الطيران — هي تختبر أدوات نشر Python وقواعد بيانات من مشروع آخر. يبدو أنها نُسخت بالخطأ. لا يوجد أي ملف

في `*.test.ts` أو `*.spec.ts` كامل المستودع.

CRITICAL • لا اختبارات و لا CI

لا Jest، لا Vitest، لا Playwright، لا Cypress، لا React Testing Library. لا يوجد سكربت test في package.json. لا يوجد مجلد .github/workflows — صفر CI/CD. كل انحدار (regression) سيكتشف فقط بعد الإنتاج بوقت طويل. هذا يلغي أي ادعاء بمستوى الإنتاج.

الحد الأدنى المطلوب للمستوى الدولي:

♦ **Unit tests** لـ `physics.ts` (حالات الـ stall، تأثير الأرض، التناوب).

♦ **Unit tests** لـ `weather.ts` (انتقال الحالات، الـ gusts).

♦ **Unit tests** لـ `achievements.ts` (فك/ربط الـ localStorage).

♦ **Component tests** لـ HUD (render مع snap وهمي).

♦ **E2E tests** لـ Playwright: إقلاع، هبوط، إنجاز.

♦ **CI workflow**: lint + type-check + test + build على كل PR.

♦ **Coverage gate** $\geq 60\%$ للبدائية، $\leq 80\%$ للإنتاج.

11 • SECURITY & LEGAL

40/100

الأمان والامتثال القانوني

لا ثغرات XSS واضحة، لكن غياب الرخصة والـ headers أمر خطير

من جهة الإيجابيات: لا يوجد `dangerouslySetInnerHTML` أو `eval()` أو `new Function()`. لا مفاتيح API في الكود. الـ localStorage يخزن تقدّم اللاعب فقط (لا PII). الطلبات الحية تستخدم HTTPS. لا قاعدة بيانات. لا محتوى مرفوع من المستخدم. هذه هي "بديهيات" الأمان الأساسية وهي متوفرة.

CRITICAL • لا ملف LICENSE

README يذكر "MIT" في السطر 185، لكن لا يوجد ملف **LICENSE في المستودع**. هذا يعني أن المشروع *جميع الحقوق محفوظة افتراضياً* (All Rights Reserved) قانونياً، ولا يمكن لأحد إعادة استخدامه قانونياً. **هذا حاجز قانوني صريح أمام أي إعادة توزيع دولية.**

MEDIUM • لا رؤوس أمن

لا CSP، لا X-Frame-Options، لا X-Content-Type-Options، لا Referrer-Policy. لا Permissions-Policy في `next.config.ts`. التطبيق يمكن تضمينه في iframe من أي موقع (clickjacking risk).

MEDIUM • لا timeout على fetch

طلبات الـ `live weather` و `elevation g` لا تستخدم `AbortController`. إذا توقف Open-Elevation (المعروف ببطله)، يبقى الـ spinner للأبد. لا `retry`، لا `exponential backoff`.

MEDIUM • لا تحقق من شكل الاستجابة

لا JSON من Open-Meteo يُعامل ك any ضمناً. إذا عاد wind_speed_10m كنص أو null، ينتج NaN ينتشر في الفيزياء. يجب schema validation (مثل zod).

— يُحذف أو يُؤمّن. "Hello, world!" placeholder api/route.ts

لا weather لل proxy server-side — عرض API usage للعامة.

لا توقيع على بيانات localStorage — قابل للتلاعب (low impact هنا).

12 • BRANDING & PWA

28/100

الهوية والعلامة التجارية

الشعار عام، لا PWA، لا SEO اجتماعي — حضور رقمي ضعيف

الشعار الحالي public/logo.svg (1 KB) هو حرف "Z" في مربع — ليس له أي علاقة بالطيران. يبدو كشعار تقني عام. لا يوجد favicon.ico، لا apple-touch-icon، لا icon-512.png / icon-192.png. المتصفحات التي لا تدعم SVG favicon تُظهر أيقونة افتراضية.

HIGH • لا PWA manifest

لا يوجد manifest.json أو webmanifest.manifest. التطبيق غير قابل للتثبيت ك app على الجوال أو سطح المكتب. لا theme-color، لا display: standalone، لا icons array.

HIGH • لا Open Graph ولا Twitter Cards

عند مشاركة الرابط على Twitter / Facebook / LinkedIn / Discord، تظهر بطاقة بيضاء فارغة — لا عنوان، لا وصف، لا صورة. هذا يقتل الانتشار الاجتماعي. يجب إضافة og:title, og:description, og:image (1200×630), twitter:card.

لا Next.js — sitemap.xml يدعم app/sitemap.ts أصلاً.

لا (JSON-LD VideoGame schema) structured data لمحركات البحث.

لا apple-mobile-web-app-capable لا iOS.

لا homepage, bugs, repository في package.json.

لا author في package.json.

العنوان عام: "Flight Simulator Pro" — يُفضّل "Flight Simulator Pro — Free Browser Flight Sim with Real Weather".

55/100

التوثيق والانضباط الهندسي

arcade-style, NOT a competitor to") المشروع (9.4 KB) README مكتوب بشكل جيد، منظم، وصادق في حدود المشروع ("MSFS"). يشرح القرارات المعمارية بدقة، يحتوي على جدول تحكم، اعتمادات النماذج، خريطة البنية. لكنه يحتوي على أخطاء وقائعية:

- ◆ README يقول "mission 6 g achievement 12" — صحيح.
- ◆ لكن `Menus.tsx:92` يكتب **"Mission Types · 10 Achievements 4"** — ثلاثة أرقام خاطئة في سطر واحد.
- ◆ README يقول "Real aircraft specs sourced from manufacturer/FAA data" — المواصفات مُعرّفة لكن غير مستخدمة في الفيزياء.
- ◆ جدول التحكم لا يذكر الـ reverse thrust (X / DLeft).

ملفات أساسية مفقودة · HIGH

لا **LICENSE** (درج قانونياً)، لا **CONTRIBUTING.md**، لا **CHANGELOG.md**، لا **CODE_OF_CONDUCT.md**، لا **SECURITY.md**. الإصدار 1.0.0 في `package.json` لكن لا `tags` ولا `releases` على GitHub. هذا يُعطي انطباع "مشروع هواة" بدلاً من مشروع إنتاج.

EndScreen منطقي في bug · MEDIUM

`Menus.tsx:210-213` يحسب تقييم الهبوط (PERFECT/GOOD/OK/ROUGH) حتى للمهام غير الهبوطية (السباق، القتال). للطيران الحر، `landingVSpeed = 0` فينتج `vsScore = 100` و `rating = PERFECT`. الواجهة تعرض "Touchdown VS: 0 fpm" و "Touchdown Spd: 0 km/h" لمهمة لم تهبط — مضلل.

14 · STRENGTHS

نقاط القوة الحقيقية للمشروع

ما يجب الحفاظ عليه والبناء عليه

رغم الدرجة المتوسطة، يملك المشروع **قرارات هندسية ناضجة** تميّزه عن المشاريع المماثلة. هذه النقاط يجب أن تُصان أثناء الإصلاحات لأنها الأصل الذي يُبنى عليه:

- ◆ **حلقة فيزياء fixed-timestep** بحماية spiral-of-death (سقف 30 خطوة، `dt clamp 0.5s`). هذا ناضج جداً.
- ◆ **scratch vectors** في module scope — صفر تخصيص في كل إطار (17 vector في `physics.ts`).
- ◆ **وضع توافق** للكرات المدمجة (Intel HD 630) — قرار استثنائي لإمكانية الوصول للأجهزة الضعيفة.
- ◆ **InstancedMesh** للأضواء المدرج (مكالمة رسم واحدة لكل لون).
- ◆ **صوت إجرائي** بطبقتين (rumble + turbine) عبر Web Audio API — لا ملفات صوتية.
- ◆ **تضاريس حقيقية** من Open-Elevation API — اختيار المطار يُغيّر الـ terrain فعلياً.
- ◆ **طقس حي** من Open-Meteo (مجاني، بلا API key) — قرار ذكي.

- ◆ تحديث HUD مقتصد على 30 Hz بدل 60 — توفير أداء.
- ◆ `dynamic import` مع `ssr: false` لتجنّب SSR لـ WebGL.
- ◆ `disposing` صحيح في `Environment.ts` — يثبت أن الفريق يعرف القاعدة لكنه لا يطبقها بشكل متّسق.
- ◆ README صادق في حدود المشروع وحدوده.
- ◆ نماذج `CC0/CC-BY` مع `attribution` — امتثال قانوني صحيح.

15 • ROADMAP

خارطة الطريق الإصلاحية — 90 يوماً للعالمية

مرتبة حسب الأولوية والاعتماديات

الخطّة التالية مصمّمة للوصول إلى الدرجة **100/+80** خلال 90 يوماً، بترتيب يراعي الاعتماديات التقنية. كل مرحلة تفتح الباب للتالية، ولا يمكن تخطّي المرحلة الأولى بأي حال لأنها تصحّح الأساس الذي يقوم عليه كل شيء آخر.

المرحلة الحرجة — إصلاح الأساس

1. إضافة ملف LICENSE (MIT) — يحل المشكلة القانونية فوراً.
2. ربط `aircraft-config.ts` بـ `physics.ts` — يُمرّز الـ `config` إلى `stepFlight`.
3. تصحيح أرقام `Menus.tsx:92` (6 missions / 12 achievements).
4. إصلاح `bug` تقييم الهبوط للمهام غير الهبوطية.
5. حذف الـ 3 `shell scripts` الغريبة من `/tests`.

المرحلة العالية — الجودة والاختبارات

1. إعادة تفعيل قواعد ESLint واحدة تلو الأخرى مع إصلاح الخروقات.
2. إضافة `Vitest` واختبارات `unit` لـ `physics.ts` و `weather.ts` و `achievements.ts`.
3. إضافة `dispose()` لـ `Airplane` و `Airport` واستدعاؤها من `FlightEngine.dispose()`.
4. إصلاح تسريب الـ `geometry` عند الـ `swap` الـ `GLB`.
5. إضافة `lint + typecheck + test + build` لـ `github/workflows/ci.yml`.
6. إضافة `AbortController + timeout` لـ `live weather fetches`.

المرحلة المتوسطة — العولمة وإمكانية الوصول

1. إضافة `next-intl` واستخراج 120- نصاً إلى ملفات ترجمة (en, ar, es, fr, zh).
2. توجيه لغوي / `[locale]` + `middleware`.
3. تحويل `Tailwind` إلى `logical properties` لدعم RTL.
4. قبّل وحدات (متروس / إمبراطوري / بحري).
5. `aria-live` للتنبيهات، `aria-hidden` للليموجي، `<kbd>` للضغطات.
6. `prefers-reduced-motion` + أوضاع تباين عالي + بدائل لونية.

المرحلة النهائية — الهوية و الإنتاج

1. تصميم شعار طيران احترافي + `favicon.ico` + `apple-touch-icon` + `icons array`.
2. `manifest.webmanifest` + PWA installability.
3. `Open Graph` + `Twitter Cards` + صورة 630×1200.
4. `sitemap.ts` + JSON-LD `VideoGame` schema.
5. رؤوس أمان في `(CSP, X-Frame-Options, Permissions-Policy)` لـ `next.config.ts`.
6. `Three.js` + `preload` + `code-splitting` لـ `GLBs`.
7. `CONTRIBUTING.md` + `CHANGELOG.md` + `SECURITY.md`.

النتيجة المتوقعة • PROJECTED OUTCOME

عند إكمال المراحل الأربع، الدرجة المتوقعة: **82-87/100** — جاهز للنشر الدولي المباشر، مؤهل للمشتريات الحكومية والتعليمية، قابل للمنافسة في فئة محاكيات المتصفح بجوار GeoFS. التكلفة التقديرية: ~360 ساعة عمل مهندس واحد، أو 3~ أسابيع فريق من 3.

الحكم النهائي والتوصية

مشروع **Flight Simulator Pro** هو عمل طموح لفريق/فرد يفهم Flight simulation بشكل حقيقي، ويملك قرارات هندسية ناضجة في الأماكن الصعبة (الفيزياء، الأداء، إدارة GPU). الأساس المعماري صلب وقابل للتوسعة. هذا ليس

مشروع هوام مبتدى — بل مشروع واعد يحتاج "نضجاً إنتاجياً" (production maturity) لإكمال المسافة الأخيرة نحو العالمية.

لكن في صورته الحالية (الدرجة 52/100)، لا يمكن التصنيف كمشروع جاهز للعالمية، للأسباب التالية مجتمعة:

- ◆ **خلل وظيفي حرج:** مواصفات الطائرات لا تؤثر على الفيزياء — الادعاء التسويقي غير صحيح.
- ◆ **غياب كامل للعولمة:** لا يمكن أن يكون "عالمياً" مشروعاً لا يدعم سوى الإنجليزية.
- ◆ **صفر اختبارات و صفر CI:** لا يمكن ضمان الجودة دون safety net.
- ◆ **إيقاف قواعد ال Lint:** يفتح الباب للانحدار الصامت.
- ◆ **لا ملف LICENSE:** حاجز قانوني صريح.
- ◆ **تسريبات ذاكرة** في فئتين أساسيتين.

RECOMMENDATION • التوصية

تأكيد نشر الإنتاج (Production Deployment) قبل الإكمال: مرفوض.
النشر ك Beta عام: مقبول مع إضافة ملف LICENSE فوراً وإصلاح physics bug.
بعد إكمال P0 + P1 (40 يوماً): يمكن النشر ك RC مع تصنيف "Early Access".
بعد إكمال P0–P3 (90 يوماً): جاهز للإصدار 1.0 الدولي مع شارة "Global Ready".

التقييم محايد، اعتمد على الكود الفعلي لا على الادعاءات. الفريق مدعو لمعالجة النقاط الدرجة أولاً — كل إصلاح في P0 يرفع الدرجة بـ 5-8 نقاط، وكل إصلاح في P1 بـ 3-5 نقاط. المسافة إلى العالمية قابلة للقطع إذا حددت الأولويات.

END OF AUDIT REPORT

العالمية قرار، لا صدفة

الفرق بين المشاريع الجيدة والمشاريع العالمية ليس في الطموح، بل في الانضباط
الهندسي على التفاصيل التي لا يراها المستخدم النهائي: الاختبارات، الرخصة، إدارة
الذاكرة، إمكانية الوصول، العولمة. كل تفصيلة في هذا التقرير هي خطوة نحو ذلك
القرار.

REPORT • FSP-AUDIT-2026-08 • METHODOLOGY • STATIC + ARCHITECTURAL
INDEPENDENT NEUTRAL AUDIT • 11 AXES • 47 RECOMMENDATIONS • 90-DAY ROADMAP